

@Override

```
public void draw() {
```

```
    System.out.println("Draw" + this.getColor() + "  
    rectangle");
```

```
}
```

@Override

```
public String getColor() {
```

```
    return "red";
```

```
}
```

```
}
```

Bài tập 1:

```
public interface Mail {
```

```
    void sendMail();
```

```
    void log();
```

```
}
```

```
public class MailImpl implements Mail {
```

@Override

```
public void sendMail() {
```

```
    System.out.println("...");
```

```
}
```

@Override


```

public void log () {
    System.out.println ("...");
}
}

```

```

public class Main
{
    public static void main (String [] args) {
        Mail Mail mail = new Mail() new MailImpl();
        mail.sendMail();
        mail.log();
    }
}

```

Bài 2:

```

public interface Shape
{
    double getArea();
    double getPerimeter();
    void draw();
}

```

```

public Ren class Rectangle implements Shape {
    private double width;
    private double height;
    public Rectangle (double width, double height)

```



```

    {
        this.width = width;
        this.height = height;
    }
    @Override
    public double getArea() {
        return width * height;
    }
    @Override
    public double getPerimeter() {
        return 2 * (width + height);
    }
    @Override
    public void draw() {
        System.out.println("Drawing Rectangle");
    }
}

public class Circle implements Shape {
    private double radius;
    public Circle(double radius) {
        this.radius = radius;
    }
}

```


@Override

```
public class Circle implements Shape {
```

```
    private double radius;
```

```
    public
```

```
    public double getArea() {
```

```
        return Math.PI * radius * radius;
```

```
    }
```

@Override

```
    public double getPerimeter() {
```

```
        return 2 * Math.PI * radius;
```

```
    }
```

@Override

```
    public void draw() {
```

```
        System.out.println("Drawing Circle");
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main (String [] args) {
```

```
        Shape r = new Rectangle (4, 5);
```

```
        Shape c = new Circle (3);
```

```
        r.draw();
```



```

System.out.println("Area:" + r.getArea());
System.out.println("Perimeter:" + r.getPerimeter());

System.out.println();
c.draw
System.out.println("Area:" + c.getArea());
System.out.println("Perimeter:" + c.getPerimeter());
}
}

```

Bài 3:

```

public class HangSX {
    String nameHangSX;
    String nameQuocGia;
    public HangSX(String nameHangSX, String nameQuocGia) {
        this.nameHangSX = nameHangSX;
        this.nameQuocGia = nameQuocGia;
    }
    public String getNameHangSX() {
        return nameHangSX;
    }
}

```



```

    }
}

public abstract class PTDC {
    String loaiPT;
    HangSX hangSX;
    public PTDC (String loaiPT, HangSX hangSX){
        this.loaiPT = loaiPT;
        this.hangSX = hangSX;
    }
    public String layNameHangSX
        return hangSX.getNameHangSX();
    }
    public void badan () {
        System.out.println("...");
    }
    public void TangToc () {
        System.out.println("...");
    }
    public void dungLai () {
        System.out.println("...");
    }
    public void abstract double layVanToc();
}

```



```
public class Plane extends PTDC {
```

```
    String loaiNhanhuan;
```

```
    public Plane (String loaiNhanhuan, HangSX hangSX)  
    {
```

```
        Super ("Plane", hangSX);
```

```
        this.loaiNhanhuan = loaiNhanhuan; }  
    public void catCanh () {
```

```
        System.out.println ("Plane cat canh");
```

```
    }  
    public void haCanh () {
```

```
        System.out.println ("Plane ha canh");
```

```
    }  
    @Override
```

```
    public double layVanToc () {
```

```
        return 800;
```

```
    }  
}
```

```
}
```

```
public class Oto extends PTDC {
```

```
    String loaiNhanhuan;
```

```
    public OtoOto (String loaiNhanhuan, HangSX  
    hangSX) {
```

```
        super ("Xe Oto", hangSX);
```



```
    this.loaiNhuenHien = loaiNhuenHien;  
}
```

```
@Override
```

```
public double LayVanToe () {  
    return 120;  
}
```

```
}
```

```
public class XeDap extends PTDC {
```

```
    public XeDap (HangSX, hangSX) {
```

```
        super ("Xe Đạp", hangSX);
```

```
    }
```

```
@Override
```

```
public double LayVanToe () {
```

```
    return 25;
```

```
}
```

```
}
```

```
public class Main {
```

```
    public static void main (String [] args) {
```

```
        HangSX h1 = new HangSX ("Toyota", "Nhac");
```

```
        HangSX h2 = new HangSX ("Boeing", "My");
```

```
        Oto Oto oto = new Oto ("Xe máy", h1);
```



```

Plane mb = new Plane ("Jet", h2);
XeDap xd = new XeDap (h1);
System.out.println ("Hãng " + oto.layNameHãngSX());
System.out.println ("Vận tốc Oto: " + oto.layVanToc());
System.out.println ("Vận tốc ở plane: " + mb.layVanToc());
System.out.println ("Vận tốc xe đạp: " + xd.layVanToc());
}
}

```

Bài Tập 4:

```

public class Car {
    protected String color;
    protected double speed;
    public Car (String color) {
        this.color = color;
        this.speed = 0;
    }
    public String get Color () {
        return color;
    }
}

```



```

        public double getSpeed() {
            return speed;
        }
    }
}

```

```

public class BMW extends Car {
    public BMW(String color) {
        super(color);
    }
}

```

@Override

```

    public String toString() {
    public void accelerate() {
        speed += 20;
    }
}

```

@Override

```

    public String toString() {
        return "BMW - color:" + color + ", Speed:"
            + speed;
    }
}
}

```

```

public class Civic extends Car {
    public Civic(String color) {
        super(color);
    }
}

```


@Override

```
public void accelerate() {
```

```
    speed += 15;
```

```
}
```

@Override

```
public void stop() {
```

```
    return "Civic - Color: " + color + "
```

```
    Speed: " + speed; }
```

```
}
```

```
public class Main {
```

```
    public static void main (String [] args) {
```

```
        BMW b = new BMW ("Black");
```

```
        Civic c = new Civic ("Red");
```

```
        b.accelerate();
```

```
        c.accelerate();
```

```
        System.out.println(b);
```

```
        "
```

```
    }
```

```
}
```